Technical University Munich

School of Engineering and Design

Report for Deep Learning for PDEs in Engineering Physics

Jorge Gracia Noguero

03801539

Supervisors: Dr. Yaohua Zang & Vincent Scholz

July 13, 2026

Contents

Introduction

Advances in machine learning and the increasing availability of high-performance computing hardware have enabled new approaches to solving partial differential equations (PDEs), complementing traditional numerical methods such as the Finite Difference Method (FDM) and the Finite Element Method (FEM). While these classical techniques can provide highly accurate solutions, they often become computationally expensive when repeated simulations or inverse analyses are required. Deep learning methods offer an alternative framework capable of either incorporating the governing physics directly into the training process or learning solution operators from data. [3]

The theoretical foundation for these approaches lies in the ability of neural networks to approximate complex nonlinear functions. Building on this property, several machine-learning frameworks have been developed for scientific computing, including Physics-Informed Neural Networks (PINNs), Deep Ritz methods, Fourier Neural Operators (FNOs), and Deep Operator Networks (DeepONets). These approaches have demonstrated promising results in forward simulation, inverse parameter identification, and surrogate modeling tasks across many areas of engineering and physics. [4]

This report investigates the application of deep learning techniques to four different PDE-related problems studied during the Deep Learning for Partial Differential Equations course at the Technical University of Munich. The problems include inverse parameter reconstruction in elastostatics, pressure-field prediction in heterogeneous porous media, surrogate modeling of heat conduction, and traffic-flow prediction based on Burgers' equation.

Implementation

Each problem will be solved with a baseline using the techniques used in class for similar problems, that is, problems that may or may not be in the same dimensional space but use the same method to that that was chosen for the problem. This solution will be presented in the Jupyter notebook of its corresponding folder of the GitHub Repository. However this baseline is **not** considered the latest solution, a different standalone python script will be provided also within the corresponding folder of the repository where the tests for the problem (different combination of hyperparameters/MLPs) are carried out and will hold the solution with the lowest L^2 relative error.

The experiments have been conducted in the following rig:

- CPU: AMD Ryzen 5 7600 X
- GPU: NVIDIA RTX 4060 8GB VRAM 3072 CUDA Cores
- RAM: 16GB DDR5 4800MT/s
- Python Version: 3.13.2
- PyTorch Version: 2.11.0+cu126

Chapter 1

Problem A: Recover Young's modulus in 1D Elastostatics

1.1 Problem Description

Consider a rod made of linearly elastic material subjected to some load. Static problems will be considered here, by which is meant it is not necessary to know how the load was applied, or how the material particles moved to reach the stressed state; it is necessary only that the load is applied slowly enough so that the accelerations are zero, or that it was applied sufficiently long ago that any vibrations have died away and movement has ceased.

The equation governing the static response of the rod is:

$$-\frac{d}{dx}\left(k(x)\frac{du}{dx}\right) = f, \quad x \in (0, L)$$

where

- $u(x)$: Displacement field of the rod
- $k(x)$: Young's modulus
- $f = 9.81$: Body force per unit length (e.g., gravity)
- $L = 1$: Length of the rod

We consider the fixation of both sides of the rod, which leads to the following boundary conditions: $u(0) = u(L) = 0$

Task: Recover the Young's modulus $k(x)$ from the observation of displacement field $u(x)$ In this task, the Young's modulus $k(x) > 0$ of the rod is unknown. However, we observe the displacement field u_{obs} (contaminated by noise with noise level $\sim 5\%$) on a set of randomly placed sensors x_{obs} (with size $N_{obs} = 500$).

The goals

- Please select a suitable deep learning method for solving this inverse problem to recover the Young's modulus $k(x)$, and explain the reason for using it
- Report your setups for the implementation, such as network structure, activation function, optimizer (with learning rate), epoch (with batch size), loss weights, and other tricks that are used for improvement.

- Compute the L^2 relative error (on testing dataset) at each training epoch and plot the "'Error vs. epoch"' curve (and report the final error). The L^2 relative error between the prediction $k_{pred}(or\ u_{pred})$ and the truth $k_{true}(or\ u_{true})$ is defined as follows:

$$error = \sqrt{\frac{\sum_i^n |k_{pred}(x_i) - k_{true}(x_i)|^2}{\sum_i^n |k_{true}(x_i)|^2}}$$

- Plot the predicted solution (and the ground truth reference) and the pointwise absolute error using separate figures with *matplotlib*.

Dataset

- *x_obs*: the observation sensors.
- *u_obs*: the observed displacement field u (contaminated by noise)
- *x_test*: the locations where the ground truth is evaluated (Used for computing error and should **not** be used for training)
- *k_test*: the ground truth reference for Young's modulus (Used for computing error and should **not** be used for training)
- *u_test*: the ground truth reference for displacement field (Used for computing error and should **not** be used for training)

1.1.1 Observations before moving forward

- The problem is inverse, the solution calls to find k(x) and not the displacement.
- The problem works with one dimensional functions.
- The physics equation is known.

1.2 Methodology

1.2.1 The method selection

The selected method is a Physics-Informed Neural Network (PINN). A PINN is a DNN that doesn't learn from labeled data but is rather forced to try and obey a physics law. Because in this problem the physics formula is provided all the necessary conditions to implement a PINN are fulfilled. This however is only true for forward problems, this particular one is an inverse problem that needs collocation points (the training/labeled data) in order to be solved. On top of that the problem describes certain conditions that can be used to improve the results of a standard PINN, such as boundary conditions and a positive Young's modulus.

While other methods could try to approximate the function using only the labeled data not enough labeled points are provided as it would be otherwise needed to make an approximation to the real function

1.2.2 The loss function

The total training loss is a weighted sum of the loss compared to the observed term and a physics (PDE-residual) term:

$$\mathcal{L} = w_{\text{OBS}}\,\mathcal{L}_{\text{OBS}} + w_{\text{PDE}}\,\mathcal{L}_{\text{PDE}} \tag{1.1}$$

The *data loss* is the mean-squared error between the predicted and observed displacement at the $N_{obs} = 500$ sensor locations:

$$\mathcal{L}_{\text{OBS}} = \frac{1}{N_{obs}} \sum_{i=1}^{N_{obs}} \left(u_\theta(x_{obs,i}) - u_{obs,i}\right)^2 \tag{1.2}$$

The *PDE residual loss* enforces the governing ODE at N_{col} collocation points $x_{c,j} \in (0, L)$, with derivatives obtained by automatic differentiation and the residual rescaled by f to make it $\mathcal{O}(1)$:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{col}} \sum_{j=1}^{N_{col}} \left(\frac{-\frac{d}{dx}\left(k_\phi(x_{c,j})\, u'_\theta(x_{c,j})\right) - f}{f} \right)^2 \tag{1.3}$$

The boundary conditions require no loss term, since $u(0) = u(L) = 0$ is imposed exactly by construction through

$$u_\theta(x) = x\,(L - x)\,N_\theta(x). \tag{1.4}$$

1.2.3 The network structure

Table 1.1: Architecture of the baseline two neural networks $u_\theta(x)$ and $k_\phi(x)$

Layer	Size / Neurons	Activation Function	Notes
Input Layer	1	–	Coordinate $x \in (0, L)$
Hidden Layer 1	40	Tanh	Fully connected
Hidden Layer 2	40	Tanh	Fully connected
Hidden Layer 3	40	Tanh	Fully connected
Hidden Layer 4	40	Tanh	Fully connected
Output Layer	1	–	Raw scalar output $N(x)$

Table 1.2: Output transforms applied to each network (hard constraints).

Network	Output transform	Purpose
$u_\theta(x)$	$u_\theta(x) = x\,(L - x)\,N_\theta(x)$	Enforces BC $u(0) = u(L) = 0$ exactly
$k_\phi(x)$	$k_\phi(x) = \text{softplus}(N_\phi(x)) + 0.1$	Enforces positivity $k(x) > 0$

The layer network structure has been conceived in this way to match that of the class' exercise for a similar PINN problem.

1.2.4 The code link

Click Here

1.3 Numerical Experiments

1.3.1 Experiment Setups

The baseline's hyperparameters can be seen in Table 1.3

Table 1.3: Numerical Setup for Training the PINN in PyTorch (baseline)

Parameter	Value
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Batch Size	Full-batch (all 500 observation points per step)
Collocation Points	2000 (resampled uniformly on $(0, L)$ every epoch)
Number of Epochs	20000
Learning Rate Scheduler	StepLR
Decay Rate (γ / N epochs)	0.5 / 5000
Loss Weights (w_{OBS} / w_{PDE})	1.0 / 0.1
Precision	Float32
Early Stopping	No (fixed 20000 epochs)

Most of these hyperparameters have been established with these values in order to match that of the class' exercise for a similar PINN problem.

Several techniques were used to improve the accuracy and stability of the inverse solution:

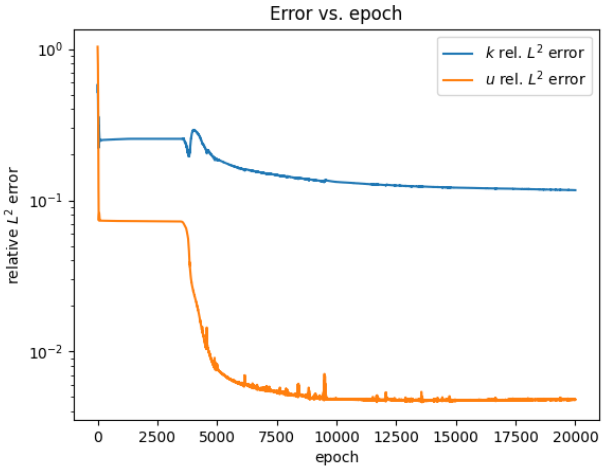
- **Hard boundary-condition enforcement.** The displacement network is written as $u_\theta(x) = x\,(L - x)\,N_\theta(x)$, so the fixed-end conditions $u(0) = u(L) = 0$ hold exactly by construction. This removes the need for a separate (soft) boundary-loss term and its associated weight, which in testing resulted in a poor performance when it came to the L^2 relative error.
- **Positivity reparametrization.** The modulus is represented as $k_\phi(x) = \text{softplus}(N_\phi(x)) + 0.1$, guaranteeing the physics requirement of $k(x) > 0$.
- **Residual rescaling.** The PDE residual is divided by the body force f so that its magnitude is $\mathcal{O}(1)$ and comparable to the data-loss term, avoiding a poorly conditioned loss balance.
- **Fresh collocation resampling.** The 2000 collocation points are re-drawn at random every epoch (a Monte-Carlo estimate of the residual integral), which enforces the physics uniformly over the domain rather than at a fixed set of points.
- **Data-dominant loss weighting.** A small residual weight (w_{PDE}) was chosen because the observations carry $\sim 5\%$ noise;a larger weight (w_{OBS}) was tested and degraded the recovered modulus (relative L^2 error $0.078 \rightarrow 0.23$).
- **Learning-rate decay.** StepLR halves the learning rate every 5000 epochs, allowing fast initial progress and finer convergence later.

1.3.2 Numerical Results

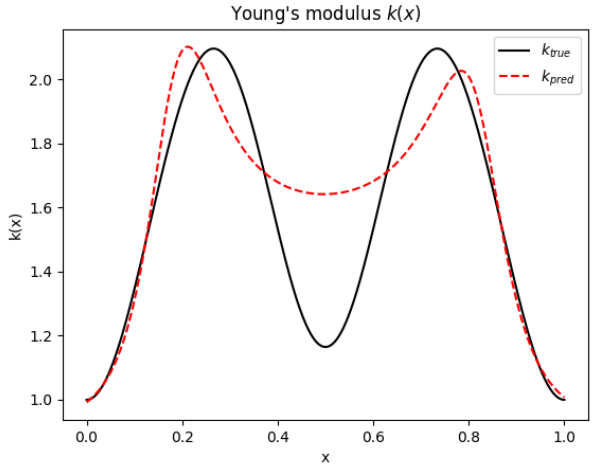
- The **time consumption** of the whole process (including data reading and plotting) required two minutes fifty-four seconds on an RTX 4060 with 3,072 CUDA cores.

- The L^2 **relative error** totaled 0.11669 for k and 0.00481 for u.

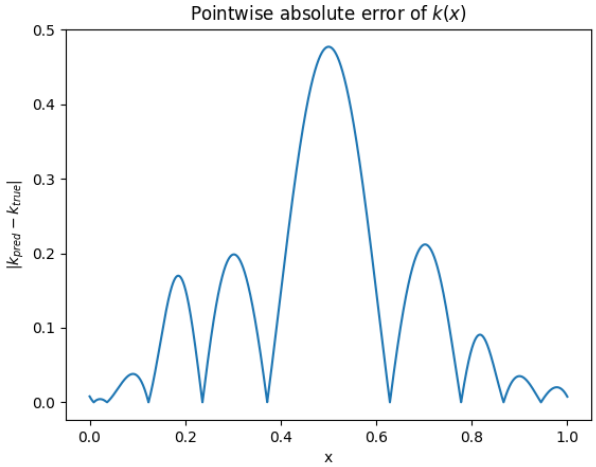
The results can be visualized in Figure 1.1



(a) Error vs. Epoch



(b) True K vs Predicted K



(c) Pointwise Absolute Error of K

Figure 1.1: Experiment results of Task 1 using the baseline model

1.3.3 Modifying the baseline

- **Changing the MLP's width and length**: Rather than having a longer but thinner network, we will now try a wider and shallower MLP with the same amount of neurons as shown in Table 1.4

Table 1.4: Architecture of the two neural networks $u_\theta(x)$ and $k_\phi(x)$

Layer	Size / Neurons	Activation Function	Notes
Input Layer	1	–	Coordinate $x \in (0, L)$
Hidden Layer 1	80	tanh	Fully connected
Hidden Layer 2	80	tanh	Fully connected
Output Layer	1	–	Raw scalar output $N(x)$

  Unsurprisingly (literature suggests deeper networks as the better alternative [4]) we get worse results for both k and u with this architecture. The L^2 **relative error** totaled 0.19208 for k and 0.0079 for u.

- **Increasing the Learning Rate** Again using the baseline we increase the learning rate from 0.001 to 0.01, accepting the risk of overshooting. This happens for u where we get much higher L^2 relative errors and the Error vs. Epoch curve is a lot more unstable due to the overshooting. See Figure 1.2

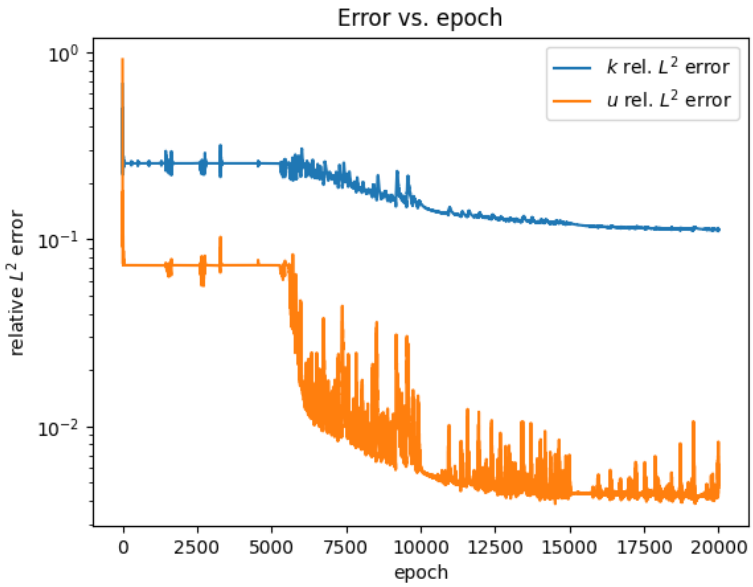


Figure 1.2: True K vs Predicted K with Increased Learning Rate = 0.01

  This change is then discarded.

- **Changing the number of Epochs** This experiment is not carried out since by looking at the graph we can see that the curves of the L^2 seem to flatten out.

- **Changing the Loss Weights** Initially the loss weight were picked with a higher emphasis on the PDE loss due to the described noise on the observed u, however, an additional

experiment was conducted using equal weights. As expected, this gives us a much poorer result. See Figure 1.3

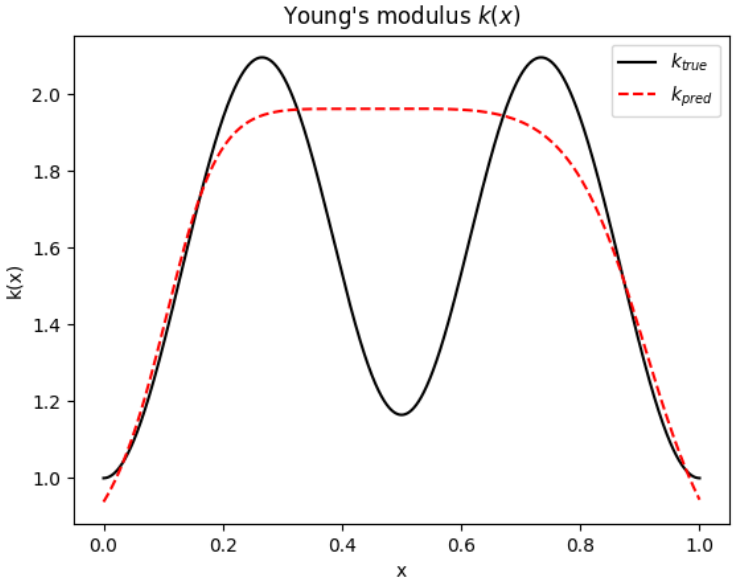


Figure 1.3: True K vs Predicted K with $W_O BD = 1$

Chapter 2

Problem B: Solve Pressure Field in 2D Heterogeneous Porous Media

2.1 Problem Description

Consider a square domain $\Omega = [0,1]^2$ representing a heterogeneous porous medium composed of two distinct material phases with significantly different permeabilities, for instance, high-permeability sandstone inclusions embedded in a low-permeability mudstone matrix. Such heterogeneous structures arise frequently in subsurface flow engineering, including groundwater modeling, hydrocarbon reservoir simulation, and fuel cell design. Near the interface between the two phases, the pressure gradient exhibits sharp transitions, making this a classical benchmark for testing the robustness of PDE solvers under non-smooth coefficients.

The permeability field $\mu(x)$ is defined as:

$$\mu(x) = \begin{cases} \mu_1 = 10, & x \in \Omega_1 \quad \text{(high-permeability phase, e.g., sandstone)} \\ \mu_2 = 2, & x \in \Omega_2 \quad \text{(low-permeability phase, e.g., mudstone)} \end{cases}$$

The specific two-phase microstructure (i.e., the geometric distribution of Ω_1 and Ω_2) is provided in the dataset. The pressure field $u(x)$ satisfies the following Darcy flow equation:

$$-\nabla \cdot (\mu(x)\, \nabla u) = f, \quad x \in \Omega$$

where:

- $u(x)$: pressure field in the porous medium.
- $\mu(x)$: spatially varying permeability field.
- $f = 0$: source term (no internal source/sink)
- $\Omega = [0,1]^2$: square computational domain

The boundary conditions are prescribed as follows, driving fluid flow from the high-pressure inlet on the left to the low-pressure outlet on the right:

$$g_D(x) = \begin{cases} 1, & x_1 = 0 \quad \text{(left boundary, high-pressure inlet)} \\ 0, & x_1 = 1 \quad \text{(right boundary, low-pressure outlet)} \\ \text{linear interpolation}, & x_2 = 0 \text{ or } 1 \quad \text{(top and bottom boundaries)} \end{cases}$$

Task: Solve the pressure field $u(x)$

In this task, the permeability field $\mu(x)$ is provided as a 128×128 pixel matrix — that is, the domain is subdivided into 128×128 cells, each with a constant permeability value. Using this permeability field, apply a suitable method introduced in the lecture to solve for the pressure field $u(x)$ in the heterogeneous porous medium.

The Goals

- Please select a suitable deep learning method for solving this forward problem to compute the pressure field $u(x)$, and explain the reason for using it.
- Report your setups for the implementation, such as network structure, activation function, optimizer (with learning rate), epoch (with batch size), loss weights, and other tricks that are used for improvement.
- Compute the L^2 relative error (on the testing dataset) at each training epoch and plot the 'Error vs. epoch' curve (and report the final error). The L^2 relative error between the prediction u_{pred} and the reference u_{true} is defined as follows:

$$\text{error} = \sqrt{\frac{\sum_i^n |u_{\text{pred}}(x_i) - u_{\text{true}}(x_i)|^2}{\sum_i^n |u_{\text{true}}(x_i)|^2}}$$

- Plot the predicted solution (and the ground truth reference) and the pointwise absolute error using separate figures with 'matplotlib'.

2.2 Methodology

2.2.1 The method selection

The selected method is the Deep Ritz method. The reason for this choice is because the permeability is piecewise constant. While a strong PINN seems like a good option at the beginning given that we have the physics function already defined, it wouldn't be a good fit since the derivatives of μ don't exist at the interfaces between the two media.

2.2.2 The loss function

The network is trained to minimize the Dirichlet energy of the Darcy problem. Since the source term is $f = 0$, the weak-form energy is

$$\mathcal{L}[u] \;=\; E[u] \;=\; \int_\Omega \tfrac{1}{2}\,\mu(x)\,\|\nabla u(x)\|^2\,dx, \qquad \Omega = [0,1]^2. \tag{2.1}$$

The minimizer of $E[u]$ over the set $\{\, u : u = g_D \text{ on } \Gamma_D \,\}$ is the weak solution of

$$-\nabla \cdot (\mu \nabla u) = 0 \tag{2.2}$$

The continuous integral is estimated by a Monte-Carlo average over the collocation points $\{x_i\}_{i=1}^N$ drawn quasi-randomly (via a Sobol sequence) from Ω. Because $|\Omega| = 1$, this average is precisely the Monte-Carlo estimate of the integral:

$$\int_\Omega f(\mathbf{x})d\mathbf{x} \approx \frac{V}{N}\sum_{i=1}^{N} f(\mathbf{x}_i) \tag{2.3}$$

$$\widehat{\mathcal{L}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \tfrac{1}{2}\,\mu(x_i)\,\big\|\nabla_x\,\hat{u}_\theta(x_i)\big\|^2. \tag{2.4}$$

There is no separate boundary-condition penalty term. On the first iterations of the code it was attempted to accomplish a good approximation to the solution by using a penalty to missing the boundary condition however it was quickly found that by hard-coding the condition within the forward function of the MLP a much closer solution with a smaller L^2 error could be achieved.

The full training objective is $\min_\theta \widehat{\mathcal{L}}(\theta)$, with the gradient $\nabla_x \hat{u}_\theta$.

2.2.3 The network structure

The network structure for the baseline model solution of this problem can be seen in Figure 2.1

Table 2.1: Architecture of the neural network (Deep Ritz residual network)

Layer	Size / Neurons	Activation Function	Notes
Input Layer	2	–	Coordinate $\mathbf{x} = (x, y) \in \Omega = [0, 1]^2$
Hidden Layer 1	64	Tanh	Fully connected
Hidden Layer 2	64	Tanh	Fully connected
Hidden Layer 3	64	Tanh	Fully connected
Hidden Layer 4	64	Tanh	Fully connected
Hidden Layer 5	64	Tanh	Fully connected
Output Layer	1	–	Raw scalar output $N(\mathbf{x})$

The size and number of layers is picked to match the recommendations from the course. The activation function is picked as Tanh since it is smoothly differentiable and zero-centered.

2.2.4 The code link

Click Here

2.3 Numerical Experiments

2.3.1 Experiment Setups

Table 2.2: Numerical Setup for Training the PINN in PyTorch (baseline)

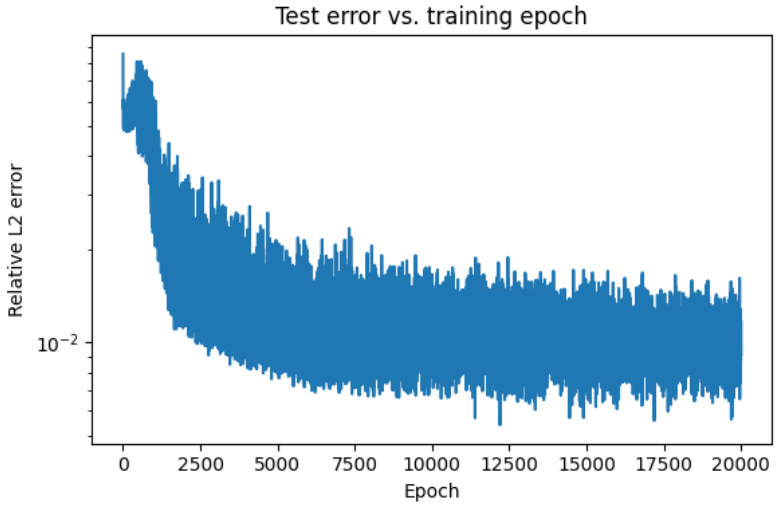
Parameter	Value
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Batch Size	40000 points
Collocation Points	40000 points
Number of Epochs	20000
Learning Rate Scheduler	StepLR
Decay Rate (γ / N epochs)	0.8 / 2000
Loss Weight w_{PDE})	None (1.0)
Precision	Float32
Early Stopping	No (fixed 20000 epochs)

Most of these hyperparameters come from the examples given in class, the choice to hold 40000 collocation points comes from a very poor result with only 10000, where the different permeabilities could not be appreciated during plotting.

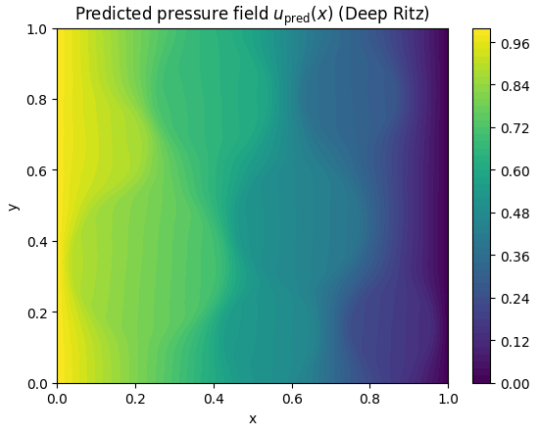
- **Energy (weak-form) loss.** The Deep Ritz method minimizes the Dirichlet energy $E[u] = \int_\Omega \frac{1}{2}\,\mu(x)\|\nabla u\|^2\,dx$ rather than the strong-form residual. This requires only first derivatives of u and uses $\mu(x)$ as a pointwise coefficient that is never differentiated, making the formulation robust to the discontinuous (piecewise-constant) permeability field, where the strong-form residual $\nabla\cdot(\mu\nabla u)$ is undefined at the phase interfaces.
- **Hard Dirichlet boundary-condition enforcement.** The pressure network is written as $\hat{u}_\theta(x) = (1 - x_1) + x_1(1 - x_1)\mathcal{N}_\theta(x)$, so the inlet/outlet conditions $u = 1$ at $x = 0$ and $u = 0$ at $x = 1$ hold exactly by construction. This removes the need for a separate (soft) boundary-loss term and its associated weight.

2.3.2 Numerical Results

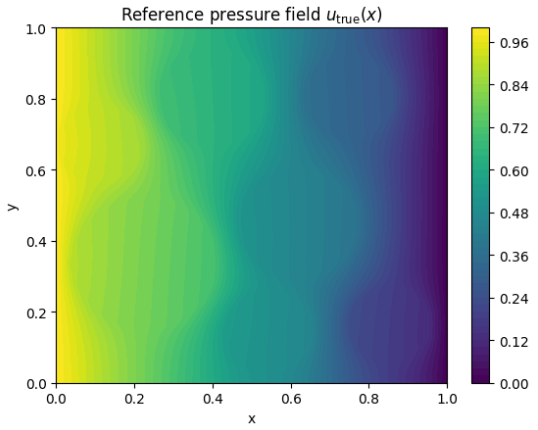
- The **time consumption** of the main loop (not accounting plotting and initial definitions) required two minutes fifty-eight seconds on an RTX 4060 with 3,072 CUDA cores.
- The L^2 **relative error** totaled 9.104e-3



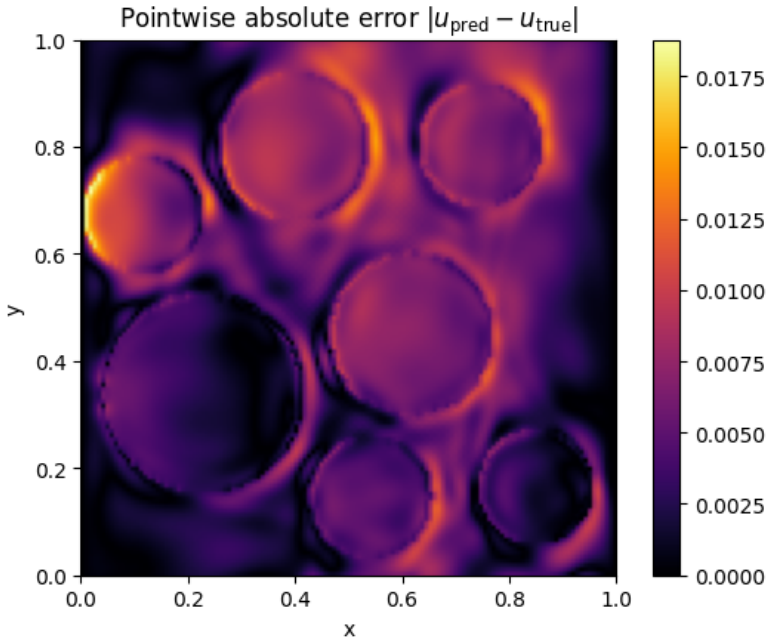
(a) Error vs. Epoch



(b) Predicted Pressure Field $u_{\text{pred}}(x)$



(c) Reference Pressure Field $u_{\text{true}}(x)$



(d) Pointwise Absolute Error

Figure 2.1: Experiment results of Problem B's baseline

2.3.3 Modifying the Baseline

- **Changing the MLP's Activation's function**: Rather than using the Tanh activation function like it was done in the exercise, some literature [1] seems to suggest that a Swish (SiLU) activation function may show similar results, so we try it here to test them. This materialized in a lower L^2 relative error with respect to the baseline of 0.007, however it's worth noting that this change nearly tripled the time it took to train the DNN.

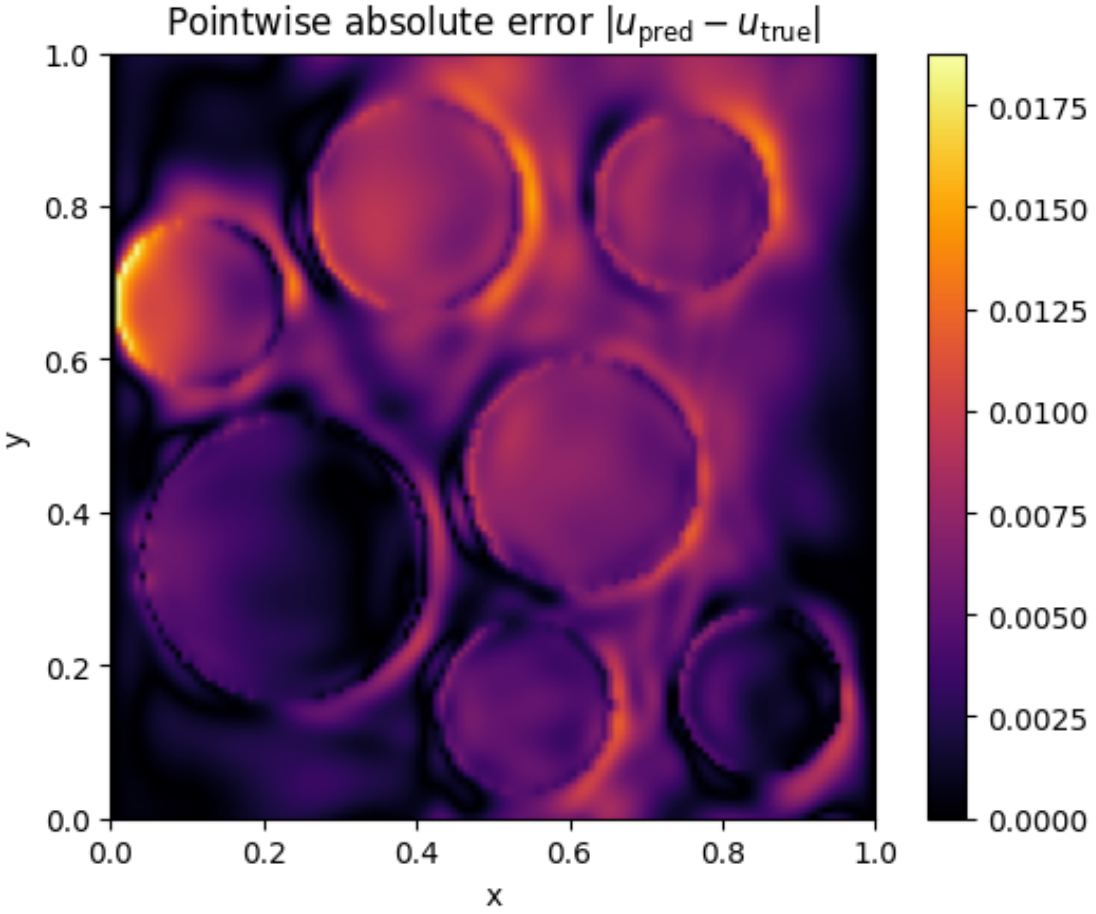
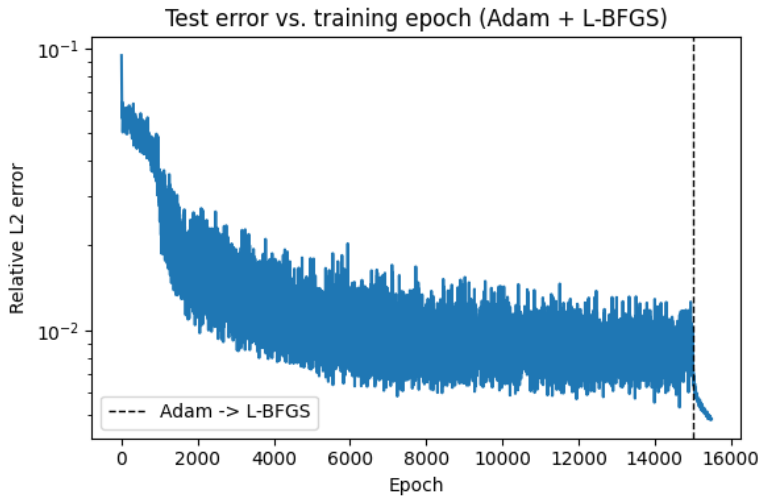
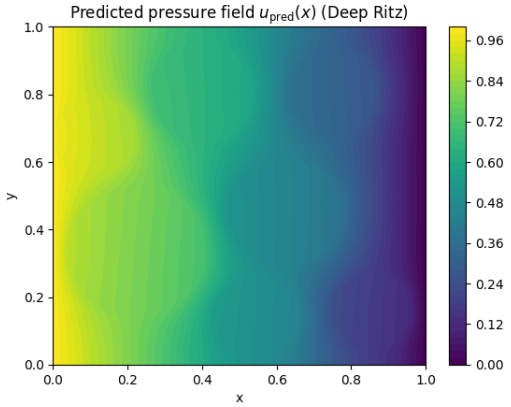


Figure 2.2: Pointwise Absolute Error with SiLU

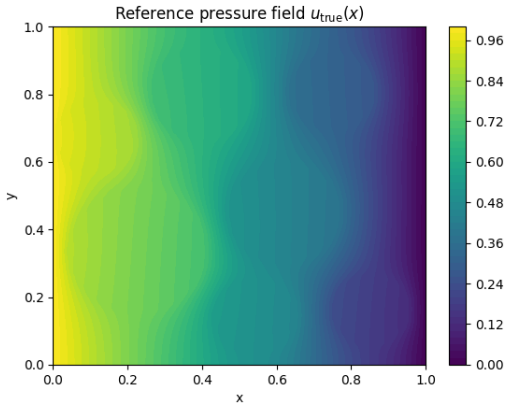
- **Two-stage Optimization Strategy**: Because of the noisy Error vs. Epoch graph it is clear that a simple scheduler of the Learning Rate it's not enough. In order to solve that a two stage optimizer method has been tested in which the Adam optimizer starts up during the training loop and later on an L-BFGS optimizer takes over. Also by passing using the Strong Wolfe line search function argument, the optimizer automatically calculates the ideal step size dynamically every epoch. As a final note, we are still using the SiLU activation function. After the run, which took around 45 minutes to complete, the final L^2 relative error was 0.00485 and led to much better results and a less noisy Error vs. Epoch graph.



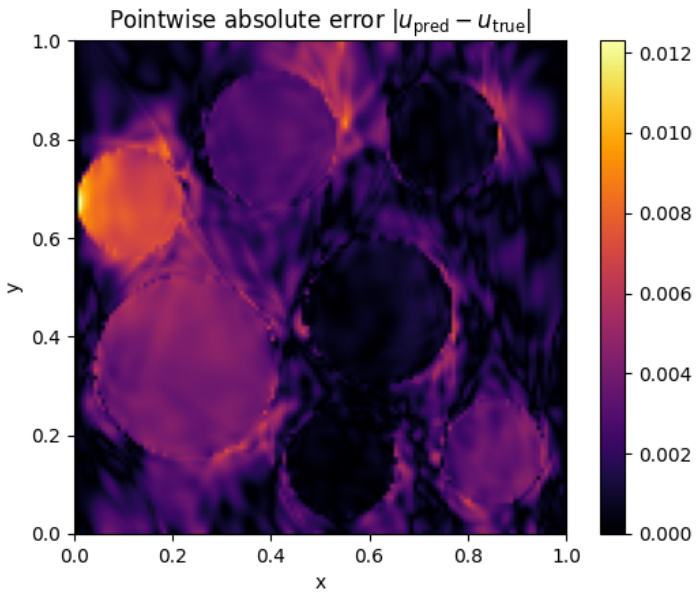
(a) Error vs. Epoch



(b) Predicted Pressure Field $u_{\text{pred}}(x)$



(c) Reference Pressure Field $u_{\text{true}}(x)$



(d) Pointwise Absolute Error

Figure 2.3: Experiment results of Problem B's with SiLU and 2 stage Optimizer

Chapter 3

Problem C: Surrogate Modeling for Steady-State Heat Conduction in Heterogeneous Materials

3.1 Problem Description

Consider the steady-state heat conduction problem in a heterogeneous solid material occupying the unit square domain $\Omega = [0, 1]^2$:

$$\begin{aligned} -\nabla \cdot (a(x, y)\nabla u) &= f, \quad (x, y) \in \Omega = [0, 1]^2 \\ u &= 0, \quad (x, y) \in \partial\Omega \end{aligned} \tag{3.1}$$

where $u(x, y)$ denotes the temperature field, $a(x, y) > 0$ denotes the spatially varying thermal conductivity of the material, and the uniform heat source is given by $f = 10$. The zero Dirichlet boundary condition models a configuration in which the boundary of the domain is kept at a fixed reference temperature.

In computational materials science, evaluating the thermal response of a large number of candidate microstructures is a central bottleneck in material screening and design workflows. Although high-fidelity Finite Element Method (FEM) solvers can produce accurate solutions, they are computationally expensive when applied repeatedly to thousands of different conductivity fields $a(x, y)$.

Task: Learning the Solution Operator $a(x, y) \mapsto u(x, y)$

The goal of this task is to make a fast prediction of the temperature field $u(x, y)$ given $a(x, y)$. To this end, a dataset of input-output pairs $\{a^{(j)}, u^{(j)}\}$ has been pre-computed using a high-precision FEM solver. The conductivity fields $a^{(j)}$ are sampled from a distribution $\mathcal{A}$ over spatially heterogeneous functions, representative of realistic microstructural variability in composite materials. Once trained, the model should accurately predict the temperature field for new, unseen conductivity samples drawn from the same distribution $\mathcal{A}$, at a fraction of the cost of a full FEM solve.

Goals:

- Please select a suitable deep learning method for this task and justify your choice.
- Report your implementation setup, including network architecture, activation function, optimizer (with learning rate), number of epochs, batch size, loss formulation, and any additional techniques used for improvement.

- Compute the L^2 relative error on the **test dataset** at each training epoch and plot the `Error vs. Epoch` curve (reporting the final error). The L^2 relative error between predictions $\{u_{\text{pred}}^{(j)}\}_{j=1}^{N}$ and ground truth $\{u_{\text{true}}^{(j)}\}_{j=1}^{N}$ is defined as:

$$\text{Relative } L^2 \text{ Error} = \frac{1}{N}\sum_{j=1}^{N}\frac{\left\|u_{\text{pred}}^{(j)} - u_{\text{true}}^{(j)}\right\|_2}{\left\|u_{\text{true}}^{(j)}\right\|_2} = \frac{1}{N}\sum_{j=1}^{N}\sqrt{\frac{\sum_i \left|u_{\text{pred}}^{(j)}(x_i) - u_{\text{true}}^{(j)}(x_i)\right|^2}{\sum_i \left|u_{\text{true}}^{(j)}(x_i)\right|^2}}$$

  where j indexes the sample and i indexes the spatial grid point.

- For the **first test instance**, use `matplotlib` to plot the following in separate figures:

  1. The input conductivity field $a(x, y)$
  2. The predicted temperature field $u_{\text{pred}}(x, y)$
  3. The ground truth temperature field $u_{\text{true}}(x, y)$
  4. The pointwise absolute error $|u_{\text{pred}} - u_{\text{true}}|$

3.2 Methodology

3.2.1 The method selection

The selected method is a Fourier Neural Operator (FNO). This choice was made with training an operator in mind which already discarded any form of PINNs or the Deep Ritz method like in the previous exercises since they would need to be retrained for every new input function (the thermal conductivity function). While a CNN could be considered, the data format is exactly what an FNO needs, a regular uniform grid so that the FFT applies. By picking an FNO over a CNN based method we improve scalability the design which scales much better with bigger sampled data, that is the FNO mixes all frequencies at once, whereas the CNN would need a larger depth to see the whole domain.

3.2.2 The loss function

The loss minimized over a mini-batch $\mathcal{B}$ of size B is the mean of the per-sample error:

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{j\in\mathcal{B}}\mathcal{E}^{(j)} = \frac{1}{B}\sum_{j\in\mathcal{B}}\frac{\left\|u_\theta^{(j)} - u_{\text{true}}^{(j)}\right\|_2}{\left\|u_{\text{true}}^{(j)}\right\|_2} \tag{3.2}$$

where $u_\theta^{(j)}$ denotes the FNO prediction with network parameters θ.

This loss function has been employed over MSE because that way it matches the evaluation metric, only being different in how it is averaged out, dividing per mini-batch training rather than the full set of tests.

3.2.3 The network structure

The network structure of the baseline solution can be found below on Table 3.1. The structure and activation function are taken from the example from class.

Table 3.1: Architecture of the neural network (Fourier Neural Operator, FNO-2D)

Layer	Channels	AF	Notes
Input	3	–	Per grid point conductivity and coordinates
Lifting (P)	40	–	Pointwise linear layer $3 \to 40$
Fourier Block 1	40	ReLU	Spectral conv + pointwise
Fourier Block 2	40	ReLU	Spectral conv + pointwise
Projection (Q_1)	128	ReLU	Pointwise linear layer $40 \to 128$
Projection (Q_2)	1	–	Pointwise linear layer $128 \to 1$

3.2.4 The code link

Click Here

3.3 Numerical Experiments

3.3.1 Experiment Setups

Table 3.2: Numerical Setup for Training the Fourier Neural Operator

Parameter	Value
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Weight Decay	1×10^{-4}
Batch Size	50 samples per step
Training Data	1000 input–output pairs $\{a^{(j)}, u^{(j)}\}$ on a 128×128 grid
Number of Epochs	50
Learning Rate Scheduler	StepLR
Loss Function	Mean per-sample relative L^2 error
Hardware	NVIDIA GeForce RTX 4060
Precision	Float32
Early Stopping	No (fixed 100 epochs)

As a special note on this section, it must be stated that the boundary condition was enforced within the definition of the neural network, although it remains unclear whether this modification fully aligns with the intended formulation of the problem.

The usage of a two stage optimizer was contemplated by reusing the code already written in Problem B, however this was deemed not necessary as the final Error vs. Epoch curve of the problem was significantly less noisy as the one of the Problem's B baseline. Also contributing to the argument of not using the two stage optimizer is that the training time of this neural network is already long enough when it comes to time (hence why only 50 epochs are used).

The optimizer, learning rate (shown in Table 3.2) and activation functions are all set to match the original paper that introduced FNOs for PDEs [2], whereas the weight decay was chosen after trying out both 1e-3 and 1e-4 with the latter giving a final lower L^2 relative error of 0.2%.

3.3.2 Numerical Results

- The **time consumption** of the main loop (not accounting plotting, data loading or object definition) required three minutes seven seconds on an RTX 4060 with 3,072

CUDA cores.

- The L^2 **relative error** totaled 0.018 (1.8%).

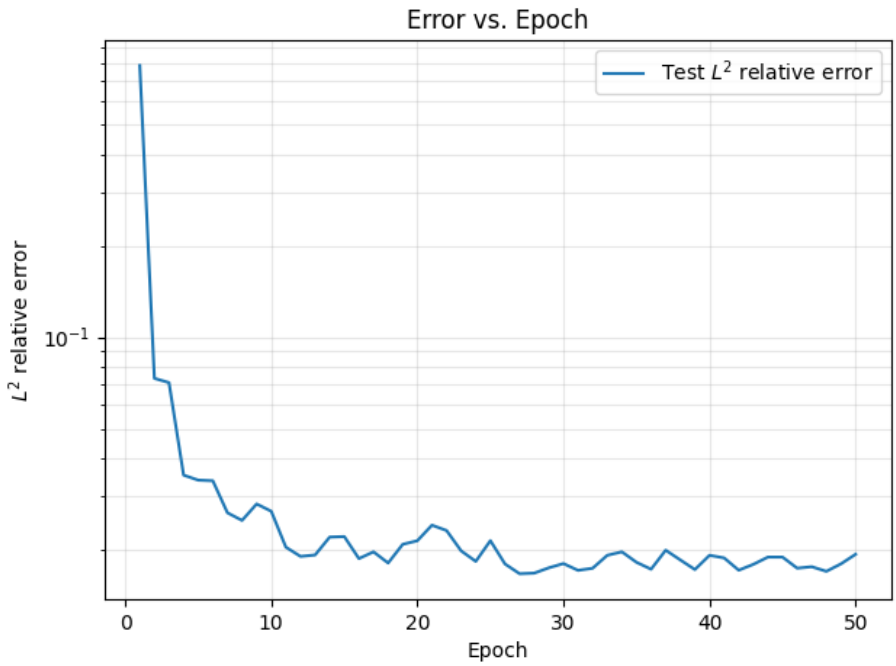
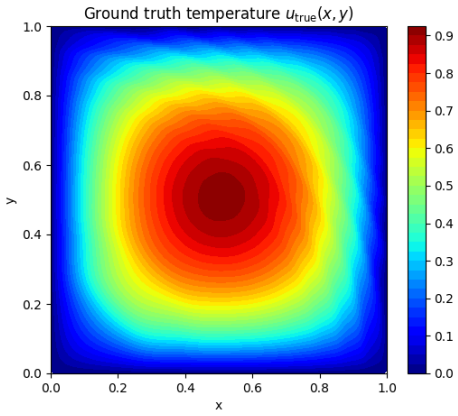
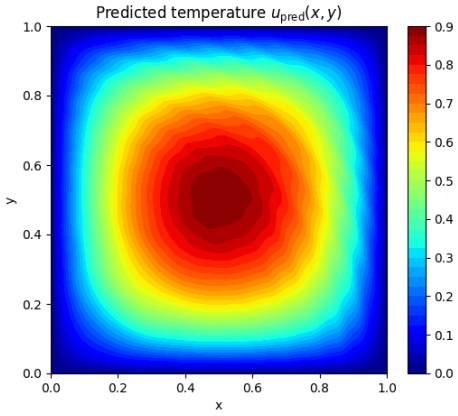


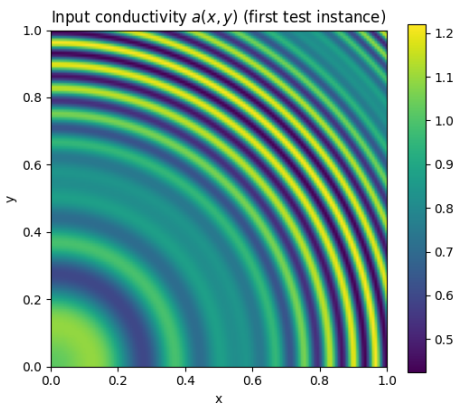
Figure 3.1: Error vs. Epoch



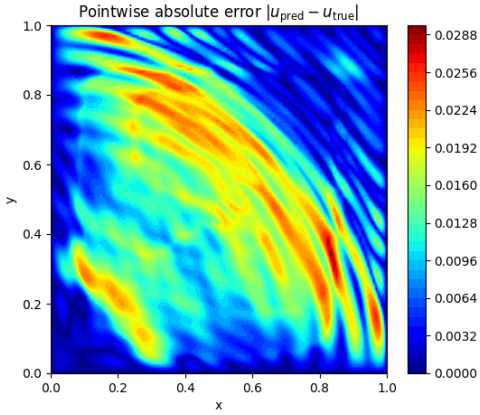
(a) Ground Truth Temperature Field



(b) Predicted Temperature Field



(c) Input Conductivity (First Test Instance)



(d) Pointwise Absolute Error

Figure 3.2: Experiment results of Problem B

Chapter 4

Problem D: Prediction of Traffic Flow Based on Burgers' Equation Model

4.1 Problem Description

Burgers' equation is a classical PDE model that captures the interplay between nonlinear convection and diffusion, making it well-suited for describing traffic flow dynamics. In this context, the convective term models the tendency for traffic to compress as vehicle density increases, while the diffusive term reflects the smoothing effect of individual driver reactions to surrounding traffic conditions.

The viscous Burgers' equation governing the evolution of vehicle velocity is:

$$u_t + uu_x = \nu u_{xx}, \qquad x \in (-1, 1),\ t \in (0, 1]$$

where:

- $u(x, t)$: Vehicle velocity field (m/s)
- $\nu = 0.1/\pi$: Kinematic viscosity coefficient (a higher value reflects more cautious, diffusive driver behavior)
- Position along the road segment
- Time

Homogeneous Dirichlet boundary conditions are imposed at both ends of the road:

$$u(-1,\ t) = u(1,\ t) = 0, \qquad t \in (0, 1]$$

Given an initial velocity profile $u(x, 0) = a(x)$, the full spatio-temporal velocity field $u(x, t)$ for $t > 0$ is uniquely determined by solving Eq. (3). In practice, however, running a high-fidelity finite difference solver for every new initial condition is computationally prohibitive. The goal of this task is therefore to learn a fast surrogate that approximates the solution operator $a(x) \mapsto u(x, t)$.

Task: predicting the velocity field $u(x, t)$ given the initial field $a(x)$

A dataset of initial conditions $a(x)$ sampled from a distribution $\mathcal{A}$ has been collected, along with the corresponding velocity fields $u(x, t)$ computed by a high-precision Finite Difference Method (FDM) solver. The dataset is **partially labeled**: only a small subset of

initial conditions is paired with its FDM solution, while a much larger set of initial conditions has no corresponding solution available. This reflects a realistic scenario in which high-fidelity simulation data is scarce and expensive to obtain. The trained surrogate model should accurately predict $u(x,t)$ for any new initial condition $a(x)$ drawn from the same distribution $\mathcal{A}$, at negligible computational cost.

Goals

- Select a suitable deep learning method for this task and justify your choice.
- Report your implementation setup, including network architecture, activation function, optimizer (with learning rate), number of epochs, batch size, loss formulation, and any additional techniques used for improvement.
- Compute the L^2 relative error on the **test dataset** at each training epoch and plot the 'Error vs. Epoch' curve (reporting the final error). The error is defined as:

$$\text{error} = \frac{1}{N}\sum_{j=1}^{N}\sqrt{\frac{\sum_i \left|u_{\text{pred}}^{(j)}(x_i,t_i) - u_{\text{true}}^{(j)}(x_i,t_i)\right|^2}{\sum_i \left|u_{\text{true}}^{(j)}(x_i,t_i)\right|^2}}$$

  where j indexes the sample and i indexes the spatio-temporal grid point.
- For the **first test instance**, use 'matplotlib' to produce four separate figures:
  1. The initial condition $a(x) = u(x,0)$
  2. The predicted velocity field $u_{\text{pred}}(x,t)$
  3. The ground truth velocity field $u_{\text{true}}(x,t)$
  4. The pointwise absolute error $|u_{\text{pred}} - u_{\text{true}}|$

4.2 Methodology

4.2.1 The method selection

The selected method for this problem is the method described in Week 10 of the course denominated Physics-Informed DeepONet. This problem satisfies the requirements for a PINO (being able to generate a mesh) however as we have already applied the FNO method for the previous problem we went with the former method for the sake of exploring the DeepONet Method.

This method exploits the hybrid learning nature of the problem. The labeled data is taken into account in order to solve the indirect signal problem. Many functions satisfy the PDE and the PDE loss can't alone pick the right one. For the rest of the unlabeled data is processed in a similar way to the PINN we solved in Problem A.

4.2.2 The loss function

PDE Loss The residual at each interior grid point is

$$r_{i,j} = (u_t)_{i,j} + u_{i,j}\,(u_x)_{i,j} - \nu\,(u_{xx})_{i,j}, \tag{4.1}$$

and the PDE loss for a batch of B input functions $\{a^{(k)}\}_{k=1}^{B}$ is the mean over the batch of the norm of the residual field:

$$\mathcal{L}_r = \frac{1}{B}\sum_{k=1}^{B} \left\| r^{(k)} \right\|_2, \qquad \left\| r^{(k)} \right\|_2 = \left(\sum_{i,j} \left(r_{i,j}^{(k)} \right)^2 \right)^{1/2}. \tag{4.2}$$

Supervised data loss Given labeled query points x with target values $u(x)$, the data-fitting term is the mean batch ℓ_2 error between predictions and targets:

$$\mathcal{L}_d = \frac{1}{B}\sum_{k=1}^{B} \left\| u^{(k)} - u_\theta\left(x^{(k)}; a^{(k)}\right) \right\|_2. \tag{4.3}$$

Total loss The training objective combines both terms with weights $\lambda_r, \lambda_d \geq 0$:

$$\mathcal{L} = \lambda_r \mathcal{L}_r \; + \; \lambda_d \mathcal{L}_d, \tag{4.4}$$

where $\lambda_d = 0$ recovers a fully physics-informed (unsupervised) training scheme.

Boundary Condition Loss Contrary like it was described in the course, the boundary condition loss is not considered within the loss function but rather is enforced within the forward function of the MLP by multiplying the raw output by a mask.

$$u_\theta(x, t; a) = \left(1 - x^2\right) \left(\sum_p b_p(a)\, \tau_p(x, t) + b_0 \right), \tag{4.5}$$

4.2.3 The network structure

Table 4.1: Architecture of the neural networks (branch and trunk)

Layer	Size	AF	Notes
Branch net $b(a)$			
Input	N_x	–	Initial condition $a(x)$ sampled at N_x sensor points
Hidden 1	128	Tanh	Linear $N_x \to 128$
Hidden 2	128	Tanh	Linear $128 \to 128$
Hidden 3	128	Tanh	Linear $128 \to 128$
Output (b)	128	–	Linear $128 \to 128$, no activation
Trunk net $\tau(x, t)$			
Input	2	–	Query point coordinates (x, t)
Hidden 1	128	Tanh	Linear $2 \to 128$
Hidden 2	128	Tanh	Linear $128 \to 128$
Hidden 3	128	Tanh	Linear $128 \to 128$
Output (τ)	128	–	Linear $128 \to 128$, no activation
Combination			
Dot product + bias	1	–	$u = \sum_p b_p\, \tau_p + b_0$
BC enforcement	1	–	Multiply by $(1 - x^2)$, hard-constrains $u(\pm 1, t) = 0$

The size of the layers has been picked to match that one of the exercise done in class regarding the same method. Regarding the activation function, matching the ReLU of the exercise was attempted however the solution did not seem to converge on the initial epochs.

4.2.4 The code link

Click Here

4.3 Numerical Experiments

4.3.1 Experiment Setups

Table 4.2: Numerical Setup for Training the

Parameter	Value
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Weight Decay	1×10^{-4}
Batch Size	50 labeled + 450 unlabeled samples per step
Training Data	200 labeled pairs $\{a^{(j)}, u^{(j)}\}$ + 1800 unlabeled samples $a^{(j)}$
Number of Epochs	500
Learning Rate Scheduler	StepLR (step size $= \lfloor \text{epochs}/6 \rfloor \approx 83$, $\gamma = 0.5$)
Loss Function	$\mathcal{L} = \lambda_r \mathcal{L}_r + \lambda_d \mathcal{L}_d$ (labeled batch) + $\lambda_r \mathcal{L}_r$ (unlabeled batch)
PDE Loss Weight (λ_r)	1.0
Data Loss Weight (λ_d)	1.0
Hardware	NVIDIA GeForce RTX 4060
Precision	Float32
Early Stopping	No (fixed 500 epochs)

4.3.2 Numerical Results

- The **time consumption** of the main training loop required two minutes and thirty-nine seconds on an RTX 4060 with 3,072 CUDA cores.
- The L^2 **relative error** was 0.45.

Looking at the plots in Figure 4.3 it appears that the predicted solution underestimates the gradients at $t = -0.25$ and $t = 0.25$ which results in the areas being a lot wider in the predicted solution than in reality. This could perhaps be solved by increasing the batch size.

4.3.3 Modifying the baseline

As using the a standard ReLU like in the class exercise did not work when it came to converging into a solution an already known activation function (Tanh) was used, however this could also be replaced by a GELU that is infinitely differentiable. Attempting this gave better results on the L^2 relative error at 0.33, see Figure 4.4.

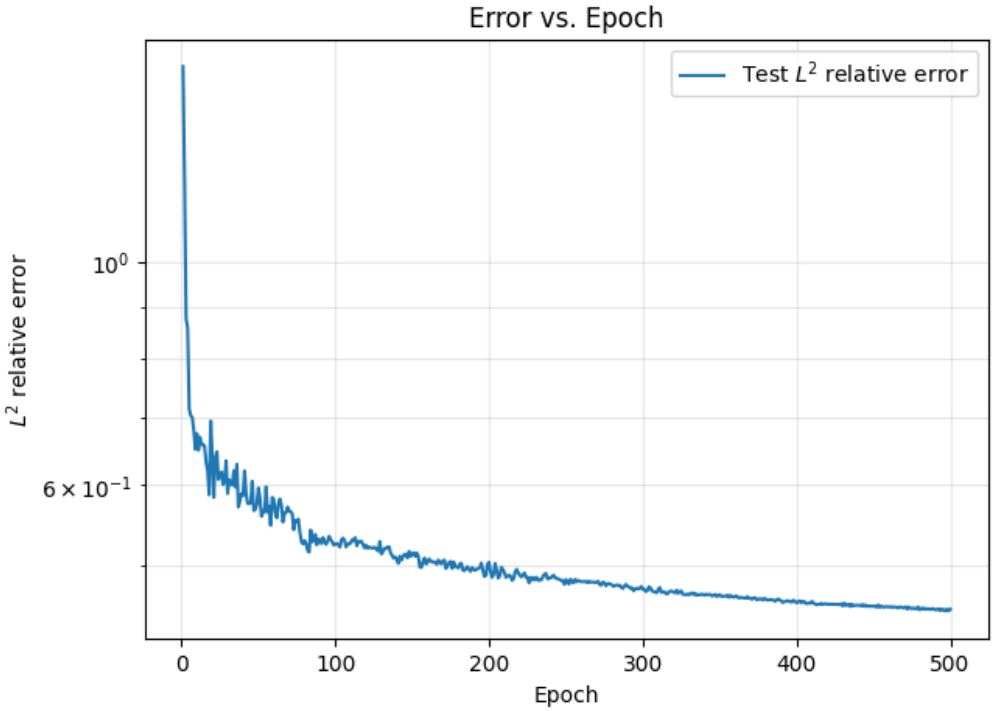


Figure 4.1: Error vs. Epoch

h

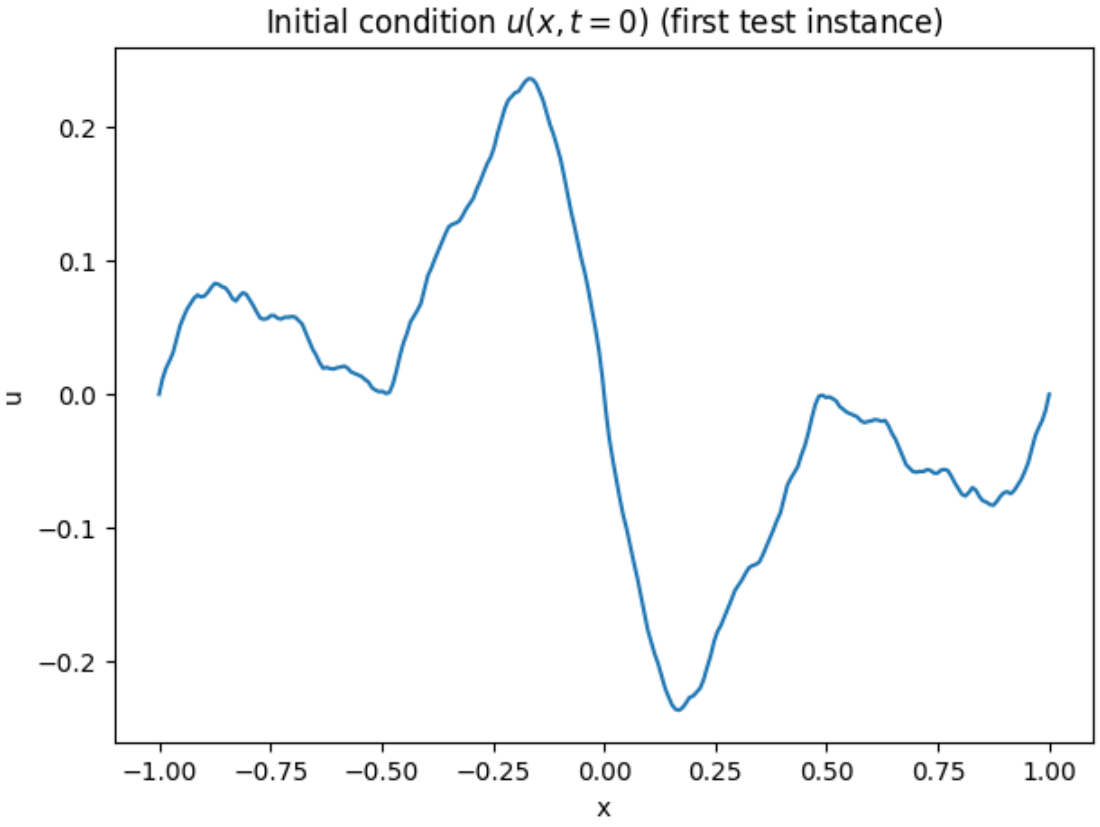
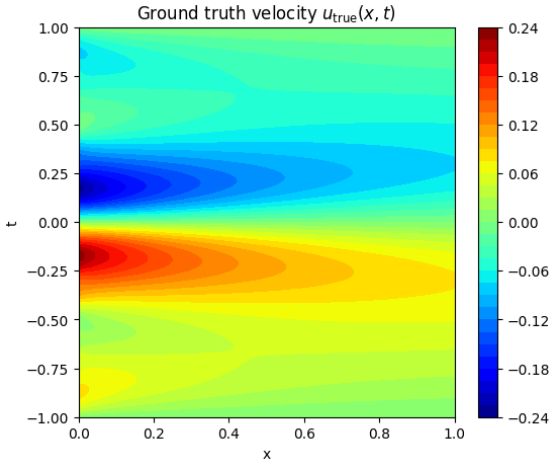
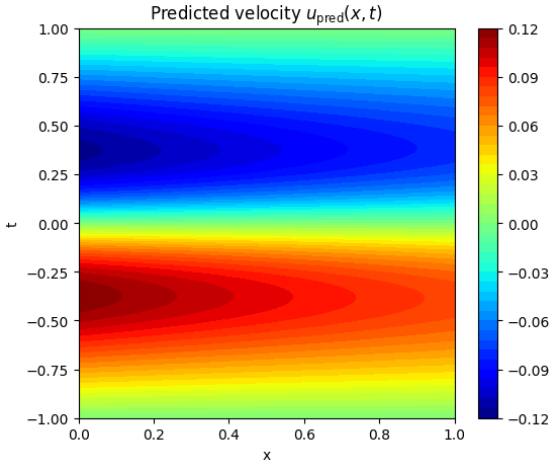


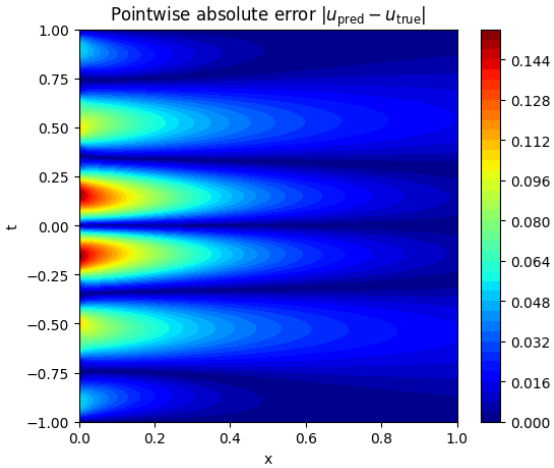
Figure 4.2: Initial Velocity Profile (First Test Instance)



(a) Ground Truth Velocity Field



(b) Predicted Velocity Field



(c) Pointwise Absolute Error

Figure 4.3: Experiment results of Problem D's baseline

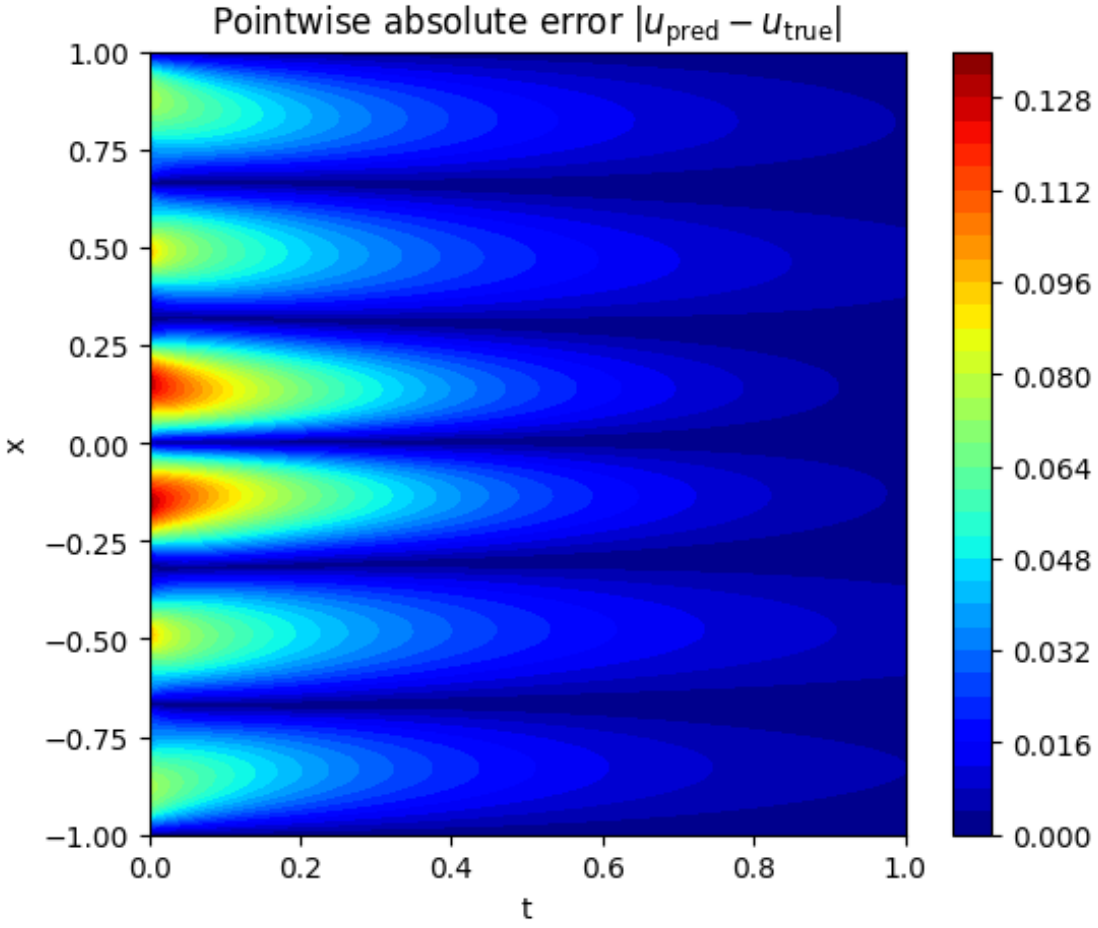


Figure 4.4: Pointwise Absolute Error with GELU activation function

Chapter 5

Conclusions

During this project four different methods have been used to tackle four different problems each with their unique variations but they all follow the same general principles.

In Problem A, the strong PINN solution performed well for the inverse parameter recovery although it showed an excessive error in the middle part of the bar, which could perhaps be improved with a two stage optimizer like in Problem B, increasing the number of epochs, with better hardware that is, and finding a better weight combination for the loss function.

In Problem B, the Deep Ritz method together with the two stage optimizer handled discontinuous coefficients effectively and cleanly which can be observed in Figure 2.3. In the Error Vs. Epoch curve it can be clearly seen that the error tends to descend before the training loop finishes which is an indicator that a higher number of epochs would have further decreased the L^2 error.

In Problem C, the FNO was the most challenging application due to the different shapes of every input. It achieved a strong operator learning performance with a low inference cost by transforming into the frequency space and performing operations there. This problem exhibited an unstable Error vs. Epoch curve 3.1 in the final epochs which could be tackled by employing the two stage optimizer described in Problem B.

In Problem D, the Physics-Informed DeepONet showed promising performance from the early stages of training, which had not happened with the previous problems, however it's worth noting that had the author had more time, another method (PINO) would have also been attempted to implement to solve this problem so the two could be compared.

Tanh appeared to be the most consistently effective activation function among those tested due to its capability to be always differentiable and being zero-centered.

During the research phase for each of the problems the author discovered how extensive the PyTorch library is. There are many optimizers, activation functions and general techniques that haven't been covered in class which would apply to each of these problems such as adaptive loss weighting and adaptive collocation sampling.

5.1 Final remarks on the course and the learnings

The course is very well organized, both the order of the lectures and its contents. Many of the topics covered were introduced in research papers published within the last decade which makes this course close to the current state of the art. I personally found this course incredibly exciting as I can see the possibilities it opened for research in optimizing revenue management for airlines which is a topic I would like to explore during my master's thesis. That said I found some of the math fundamentals hard to follow as an aerospace engineer student with

an engineering background rather than a mathematical one, however I do think that with enough time every concept can be understood.

References

[1] A. Al-Safwan, C. Song, and U. bin Waheed. Is it time to swish? comparing activation functions in solving the helmholtz equation using physics-informed neural networks, 2021. URL https://arxiv.org/abs/2110.07721.

[2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=c8P9WAb6Gkb.

[3] R. Stephany and C. Earls. Pde-learn: Using deep learning to discover partial differential equations from noisy, limited data. *Neural Networks*, 174:106242, 2024. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2024.106242. URL https://www.sciencedirect.com/science/article/pii/S0893608024001667.

[4] Y. Zang and V. Scholz. Deep learning for pdes presentation slides.